

Myeongseong Kim

Frontend Engineer · 5 yrs 7 mos experience

A frontend engineer who migrates legacy systems with zero downtime and turns decision rationale into team standards through RFCs.

010 8663 2663    maeng2418@gmail.com    github.com/maeng2418    maeng2418.github.io

Key Achievements

4-year legacy migration completed

Led the incremental Vue/Nuxt → React migration through RFC consensus. Legacy code share driven from 100% (2022) to 1.1% (2026) with zero downtime

Build 589.24s → 1.10s on cache hit

Adopted Turborepo and cleaned up build configuration. Raw measurement logs recorded in internal docs

Initial load 3.4MB → 1.5MB (55.9% reduction)

DOMContentLoaded 346ms → 121ms. Achieved by disproving an AI-suggested conclusion with instrumentation and redefining the root cause

Introduced E2E testing to a test-less monorepo

Built Playwright-based regression verification from zero and integrated it into CI

Enforced conventions through tooling, not documents

Promoted ADRs to AI skills, recurring review comments to lint rules, and verification to CI gates

Work Experience

|                                                                                     |                                              |                                                                                                                                                                 |
|-------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>NOL Universe</div> <div>(Yanolja)</div> | <div>2022.09.26 – Present (3 yrs 10 mos)</div> <div>Global B2B console where accommodation partners manage rates and inventory directly · PC/Mobile/Admin</div> |
|  | <div>IGAWorks</div> <div>(Adpopcorn)</div>   | <div>2021.07 – 2022.09</div> <div>Ad-tech data platform · SSP console renewal (NextJS→React), ad performance dashboards, Svelte landing pages</div>             |
|  | <div>Zimssa (Partner App)</div>              | <div>2020.12 – 2021.07</div> <div>Moving-service O2O hybrid app (Cordova) · JS→TS migration, Atomic Design system, component CLI</div>                          |

Skills

Core — TypeScript · React · Next.js

State & Forms   TanStack Query · React Hook Form · Zod  
Build & Test   Vite · Turborepo · pnpm · Playwright · Vitest · Storybook  
Styling & Architecture   MUI · Tailwind CSS · Emotion · FSD  
Infra   GitHub Actions · Docker/EKS

# Key Projects

---

## Incremental Vue/Nuxt → React Migration — Architecture Design & Lead

NOL Universe · Partner B2B console · 543 legacy pages in 7 repos + 10 new repos · 3 FE engineers

### Problem

The legacy codebase and new React apps had to coexist while migrating without any service interruption. The Vue/Nuxt stack was the very weakness I had flagged myself when joining in 2022.

### Decision — Migration Architecture

I first documented the migration strategy as an RFC and secured team consensus. Rather than migrating all at once, we chose to build the shared infra, auth, and routing foundation in Phase 0, then migrate domain by domain.

### Execution

Secured old/new app coexistence with nginx path routing and Istio VirtualService splitting. Large migrations were split into Parts 1–4, pairing new implementation with legacy removal; even the final Nuxt teardown was designed upfront.

### Result

Legacy PR share: 100% (2022) → 69.4% (2023) → 22.3% (2024) → 6.5% (2025) → 1.1% (2026). Completed a 4-year journey through the very stack I had once listed as a weakness.

## Performance Improvement — Verifying and Rejecting an AI Draft

Mobile web · Bundling strategy redefined

### Problem

A specific page was loading a massive 3.44MB chunk. When I handed the analysis to an LLM, it concluded that React.lazy code splitting had dramatically reduced it.

### Decision

I found the caveat the draft itself had noted — DOMContentLoaded and Load were identical before and after, and the draft admitted the numbers were "likely measured with cache applied." I re-compared keeping code splitting vs. a single bundle.

### Execution

I rejected the AI's conclusion and redefined the problem: the real issue was the root cause of the excessive payload, not splitting vs. monolith.

After removing the giant chunk I re-measured directly and chose a single bundle — accepting that every page loads 1.71MB as more predictable than the previous per-page variance.

### Result

Initial load 3.4MB → 1.5MB (55.9% reduction), DOMContentLoaded 346ms → 121ms, Load 487ms → 349ms. Evidence scope: client-side measurements from one mobile app.

## Client Log Instrumentation DevTools — Moving Verification Cost from Code to Screen

NOL Universe · 304 partner-console log definitions · 4 apps wired (MUI/NDSP) · 0 new dependencies

### Problem

304 log definitions sat scattered in 10K lines of generated code, with no way outside the code to see what was instrumented or actually landed. Dead definitions, sender-definition drift (174 files), and StrictMode double-firing accumulated.

### Decision

I moved the verification cost onto the screen. The inspector was built in vanilla DOM, free of any app design system, so it drops identically into MUI and NDSP apps; collection changes were limited to one pilot screen to protect stored data.

### Execution

Reverse-parsed the generated sender code into 5 deterministic catalog artifacts (human md, detailed json, warehouse-diff SQL, runtime meta, ref types). Built a dev-only inspector — live firing feed, 30-day volume overlay, txtpb generation — plus an auto-firing runtime where one tagging line becomes collection.

### Result

Verified byte-identical payloads between auto-firing and existing senders; production bundles grew by only 96KB of slim metadata. The Databricks PAT was isolated behind a local dev proxy (0 new dependencies), and adoption/migration moved to a convention RFC.

# Key Projects (continued)

---

## Standardizing the Team's AI Development Infrastructure

[Established an internal Claude plugin marketplace · 5 consuming repos](#)

### Problem

Development tooling was locked into the monorepo, and the boundary between personal and team assets was unclear. Leaving installation to individuals also risked required rules being omitted.

### Decision

I compared 9 alternatives — 4 repo-bound and 5 shared. Each option's trade-offs and rationale were recorded in a document. Since one bad change to shared team tooling can block everyone's work, I chose to lay down CI, permission, and convention safety nets first, then add features.

### Execution

Adopted a two-axis structure: repo-bound assets via .claude/project-plugin with a gitignore whitelist, and shared assets in a separate marketplace repo.

### Result

Launched the internal marketplace, shipped 8 plugins, rolled them out to 5 consuming repos, and folded them into onboarding docs.

## Knowledge System Redesign — LLM-Wiki Migration

[Knowledge build pipeline for a large monorepo](#)

### Problem

With one skill per domain, working the same domain across 10 sessions fragmented into 10 separate drafts.

### Decision

I recorded 3 rejected alternatives with reasons — vector DB/embeddings (markdown plus grep was sufficient), hook auto-injection (false positives and complexity), and draft-PR conversion (to keep automatic reviewer assignment).

### Execution

Designed a three-way ADD/UPDATE/NOOP verdict structure and a two-layer guard against self-proliferation.

### Result

Independent audit score from 0.52 FAIL to 0.87 PASS, evaluators at 93/85/90/92, 129 tests green.

## Global Partner Central — Sales Calendar Architecture & Data Transformation Standards

[PC·Admin for global accommodation partners · Multi-language/multi-currency · 2 FE + 2 PM·BE, PPD, QA collaboration](#)

### Problem

We had to build a new set of screens for global accommodation partners to manage rate plans, inventory, and sales status. If shared policies like currency decimal places and multilingual rate-plan labels were implemented per screen, every policy change would require hunting down and fixing every screen.

### Decision — Architecture

I pushed shared policies (currency decimals, rate-plan i18n) down into lower layers and split FSD layers so screens only compose. The guiding rule was "keep the blast radius of policy changes narrow."

The same principle governed form/request transforms via a 3-axis rule — role (validation hook / pure transform), location (co-located under features), and shape — and we codified that passthrough requests get no wrappers, preventing forced abstraction.

### Execution

Where no dedicated designer was available, I produced multiple mockups, shared them with teammates, and adopted the option chosen through feedback.

The full convention rollout was split into 6 stacked PRs, each passing type checks and lint. Since request objects are components of query cache keys, their shape and values were preserved byte-for-byte so cache behavior never changed.

### Result

Aligned the calendar, base pricing, and Daily Rate List to show identical occupancy rates and Cap values, and shipped new features behind feature flags without breaking operations. Convention compliance was fully normalized from the initial 75–80%.



# Previous Experience in Detail

---



## IGAWorks (Adpopcorn)

2021.07 – 2022.09 · Ad-tech data platform · Frontend Engineer

### Role

Full renewal of the mobile ad SSP platform (Adpopcorn) console, development of a news ad platform (Conpick), and ad performance analytics dashboards.

### Decision & Execution

The existing NextJS SSR architecture did not fit a console product, so I migrated it to an SPA, removing unnecessary server-rendering overhead. Redesigned state management with React-Query and Recoil, eliminating duplicate API calls and optimizing bundles.

For landing pages, I chose Svelte — whose framework disappears at compile time — to secure load performance.

### Result

Delivered dashboards visualizing per-campaign revenue, clicks, and impressions in real time with Recharts + SVG, and operated a zero-downtime deployment pipeline with AWS CodePipeline CI/CD and ECS/EKS container environments.



## Zimssa (Partner App)

2020.12 – 2021.07 · Moving-service O2O platform · Frontend Developer

### Role

Developed the moving-match hybrid app (Zimssa Partner) as a Cordova-based iOS/Android cross-platform app. Led the Design 2.0 renewal.

### Decision & Execution

Migrated the JavaScript codebase to TypeScript and applied lazy loading to improve initial load. Built an Atomic Design component system for UI consistency, attached a Storybook-based verification workflow, and added dark mode support.

### Result

Established code quality tooling with ESLint, Prettier, and Husky, and built a component CLI that auto-generates Atomic Design boilerplate to cut repetitive work.

# Standards Established

---

## Enforcing conventions through tooling, not documents

Believing that documents which merely hope to be read never get followed, I pushed rules down into the execution layer. ADRs were migrated to AI skills that apply automatically at work time, recurring review comments became lint rules, and build/type verification was promoted to CI gates.

## Conventions as skills — type derivation rules · empty-value notation · table cell navigation · frontmatter policy

Each convention was built as a skill instead of a document and applied across the whole team at once. When a rule changes, only the skill needs updating.

## Porting collaboration standards to new repos

Set up CODEOWNERS, PR templates, 3 GitHub Actions workflows, 3 husky hooks, and commitlint as the first PR of every new repo, so standards are enforced from the very first commit.

# Structural Improvements & Safety Nets

---

## From code to declarations — structural design reduced change points from 11 files to 1

Sales-lock criteria were scattered across 11 files, so changing one policy meant finding and editing every usage — any omission became a bug behaving differently per screen. I separated per-channel rules into policy declarations rather than code branches, consolidating them into a single point; rule changes now end with editing one declaration, leaving call sites untouched.

## Removing triple-duplicated locale types — ~20 existing call sites untouched

Multilingual text types were defined in three separate places, so fixing one broke alignment with the others. I unified them into a single definition via a module-augmentation slot pattern, designed so that ~20 existing consumers kept compiling as-is — a case of making large-scale cleanup incrementally adoptable instead of one big cutover.

## Building E2E regression verification from a zero-test state

The monorepo had no test code at all. Introducing Playwright, I went beyond just attaching the tool — login sessions and data preparation were standardized as fixtures so each test never rebuilds preconditions (2,833 new lines). Core-flow regression checks were folded into CI, running automatically before every deploy.

## Removing an auth token exposed in deployed bundles

A Sentry auth token was being shipped inside client bundle artifacts — a build-time-only value leaking into runtime output. I separated the injection path and removed it from the bundle.

# Activities & Documentation

---

## Internal technical writing

Authored Confluence docs on Playwright E2E adoption, Turborepo rollout, chart library replacement, shared component consolidation, and rate-plan domain specs. 4 release launch plans; 60+ Partner Center product/operations guides.

## RFCs & Tech Specs

React migration strategy for the partner onboarding admin (11 alternatives compared), NDSP component API consistency, and infra path-splitting design.

## Woowahan Tech Camp 3rd cohort · 2020.07 – 2020.08

8-week web development program run by Woowa Brothers. Practiced code review and pair programming.

# Education

---



**B.S. in Computer Engineering, Seoul National University of Science and Technology**  
2015.03 – 2021.02